

BTUT From First Principles

A complete walkthrough of the BTUT coordination platform — what each part computes, why it was built that way, and what it actually does when you run it. Written to be read straight through.

Repository [direncode/btut](#) **Revision** [d137615](#) **Volume** 34,341 lines · 318 files

Languages [Rust](#) · [Python](#) · [TypeScript](#) **Measurements** taken on this checkout

CONTENTS

- 1 The problem, and the trade BTUT makes**
Why coordination is expensive, and what you give up to make it linear.
- 2 The system on one page**
Every component, what talks to what, and why the repository has the shape it has.
- 3 Engine A: the algorithm everything runs on**
Five steps, three implementations, read line by line.
- 4 What Engine A does when you run it**
The closed-form solution, measured. What holds and what doesn't.
- 5 Engine B: the market kernel**
A second, unrelated algorithm — and why it exists.
- 6 The market simulator**
Order books, trader classes, crash detection, and one honest approximation.
- 7 Calibration: choosing gamma**
The one parameter that decides everything, and the search that finds it.
- 8 The service layer**
Fourteen HTTP routes, and the three shortcuts inside them.

9 The client surfaces

Two web apps, two browser engines, one SDK that isn't an engine.

10 The domain bridges

Traffic, drones, robots — and the one place the model genuinely changes.

11 Deployment and verification

Four targets, 36 Rust tests, 66 Python tests.

12 Everything that is wrong or unfinished

The complete list, with evidence.

13 What to do next

An ordered plan, cheapest and highest-value first.

SECTION ONE

The problem, and the trade BTUT makes

Before any code: what makes multi-agent coordination expensive, and what BTUT gives up in exchange for making it cheap.

Suppose you have a million autonomous things — vehicles, drones, traders — and each one's best move depends on what the others do. The honest way to model this is to write down every interaction. With N agents that is N^2 pairs, and if you go further and model the population as a continuous density evolving under a partial differential equation, solving it numerically costs more still. This is the standard picture, and it is the reason academic agent-based tools fall over in the tens of thousands. The repository's own framing points at DARPA's Mathematical Challenge 13, which asks how to coordinate very large numbers of autonomous agents efficiently.

BTUT's answer is a mean-field approximation, which is an old and respectable idea. Instead of letting agent i respond to each of its neighbours individually, you let it respond to a single number that summarises the entire population — here, the fraction of agents currently cooperating. You compute that number once per round by sweeping all N agents, then you sweep all N agents again to update them against it. Two linear passes. No adjacency matrix is ever allocated, so memory is linear too.

The whole system is built on one substitution: replace "what are my neighbours doing" with "what is everyone doing, on average, weighted by importance." Everything else in the repository is delivery around that substitution.

What you give up is real and worth naming up front. In a genuine network, an agent surrounded by defectors behaves differently from an agent surrounded by cooperators, even when the global average is identical. Mean-field erases that. Local structure — clusters, bridges, isolated pockets — stops existing. In exchange you get a model that runs on a million agents in seconds, and whose behaviour you can often characterise analytically. For some questions that trade is excellent; for others it removes the phenomenon you were trying to study.

1.1 Where the "network" went

BTUT does not throw the network away entirely. It keeps one summary statistic of it: each agent's *degree* — how many neighbours it would have had. Degrees are drawn directly from the distribution a scale-free network would have produced, without ever building the network. Agents with high degree get more weight in the global average, on the reasoning that hubs matter more.

The exponent controlling that, called τ , is one of the system's two important knobs. At $\tau = 0$ every agent counts equally. At $\tau = 1$ an agent's influence is proportional to its degree, and a handful of hubs dominate the population average. This is the repository's stand-in for network topology: not a graph, but a single number describing how unequal the influence distribution is.

WHY SAMPLE DEGREES INSTEAD OF BUILDING A GRAPH

Building a Barabási–Albert graph for a million nodes means storing several million edges and doing preferential-attachment bookkeeping during construction. Sampling from the resulting degree distribution costs one uniform random number per agent and stores one float. Since the algorithm only ever uses degree — never the identity of a neighbour — the graph would be constructed and then never traversed.

The inverse-CDF trick makes it exact rather than approximate: for a degree distribution $P(k) \propto k^{-3}$, drawing u uniformly and computing $k = m/\sqrt{1-u}$ produces exactly that distribution. It's one line, and it is mathematically the right one line.

1.2 The second knob: gamma

The other important parameter is γ , and it deserves early billing because it turns out to control almost everything. Agents in BTUT are imagined to be playing a mixture of three classical two-player games — Prisoner's Dilemma, Hawk–Dove, and Stag Hunt. γ is the cooperation bonus in the Stag Hunt component: how much better off you are if you cooperate and everyone else does too.

Stag Hunt is the game with two stable equilibria — everyone hunts stag, or everyone hunts hare — and which one you land in depends on expectations rather than on payoffs alone. That structure is what gives BTUT a *phase transition*: below a critical value of γ the population collapses to universal defection, above it the population saturates at universal cooperation, and the switch between the two is abrupt. Section 4 measures exactly how abrupt. Section 7 covers the machinery built to find that critical value automatically, which exists precisely because getting γ wrong doesn't degrade the result gracefully — it inverts it.

SECTION TWO

The system on one page

What exists, what calls what, and why 34,000 lines were needed to deliver a loop that fits on a screen.

The repository has three tiers. At the bottom is the algorithm, implemented four times. In the middle are the ways you can reach it: an HTTP service, a web application, and two client libraries. At the top are four application domains that map the abstract model onto something physical.

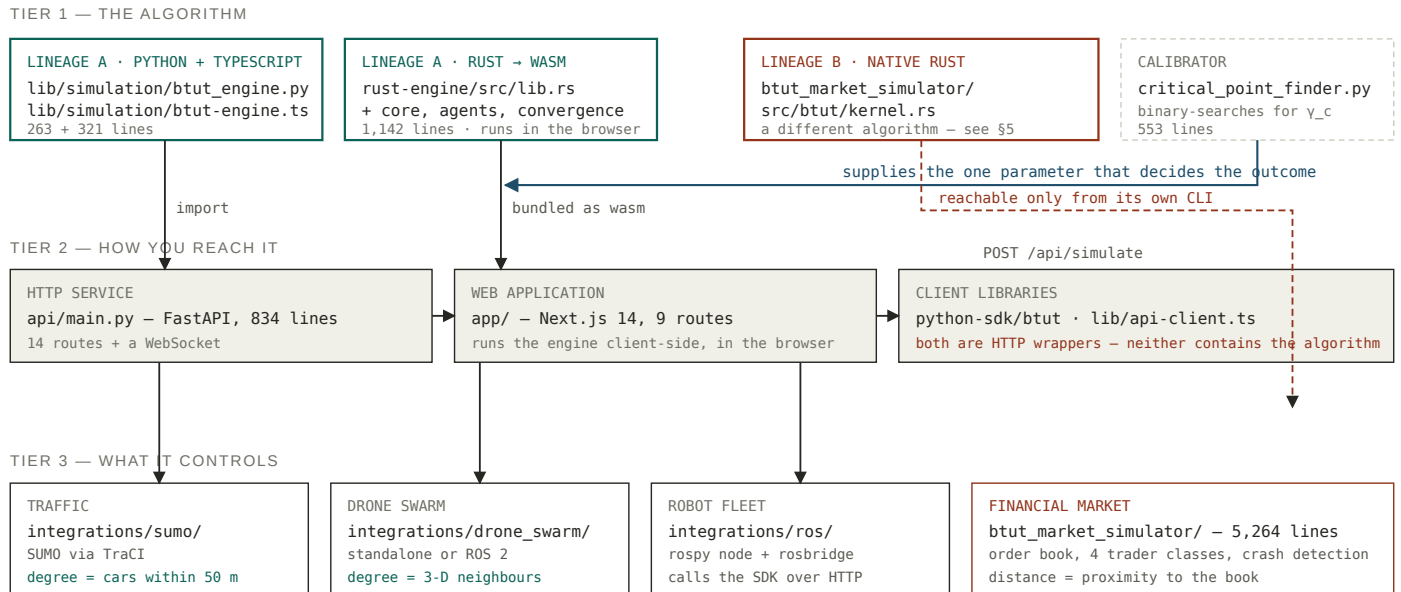


Figure 1. The whole platform. Two engine lineages sit at the top and never meet — Lineage A (teal) is what every user-facing path runs; Lineage B (rust-red) is a separate algorithm reachable only from its own command line. The calibrator is the single component whose output feeds every Lineage A run.

2.1 Why the algorithm exists four times

Writing one algorithm four times is normally a mistake. Here each copy has a defensible reason, and understanding those reasons explains most of the repository's structure.

The **Python** copy exists because the HTTP service is FastAPI and needs to call the engine in-process. It uses NumPy for setup and a plain loop for the update.

The **Rust-to-WASM** copy exists because the web application runs simulations *in the visitor's browser* rather than on a server. That is a deliberate and rather good product decision: a demo that runs client-side has no per-visitor server cost, no queue, no rate limit, and no cold start. A hundred thousand agents at sixty frames a second is not something you want to serve from a Python process.

The **TypeScript** copy exists as the fallback for when the WASM module fails to load — an old browser, a strict content policy, a failed fetch. Without it the demo page would show an error instead of a simulation.

The **native Rust** copy is the one that isn't a copy at all. It is a different algorithm, and Section 5 is about it.

WHY THIS IS DEFENSIBLE, AND WHERE IT WENT WRONG

Three ports of one algorithm across three runtimes is the standard cost of wanting the same computation on a server, in a browser, and at native speed. There is no single language that does all three well, and WebAssembly plus a JavaScript fallback is the conventional answer.

The failure isn't the duplication — it's that no test pins the three implementations to each other. If a single test ran all three on the same seed and asserted matching output, the divergence documented in Section 3.4 would have been caught the day it was introduced. Instead the three drifted, and the browser now silently runs a different model depending on whether WASM loaded.

2.2 Why there are two different algorithms called BTUT

This is the single fact that makes the repository legible, and no document in it states the fact. The market simulator's kernel shares vocabulary with Lineage A — kernel, weight, cooperation, convergence — and shares nothing else: different state, different update rule, different parameters, different files, no calls between them.

	LINEAGE A — MEAN-FIELD GAME	LINEAGE B — MARKET KERNEL
Agent state	degree, weight, strategy $\in \{A, B\}$	cooperation $\in [0,1]$ continuous
What drives it	Three mixed games producing two payoff scalars	No games. A weighted population average.
Update	adopt $\text{argmax}(U_A \cdot w_i k_i, U_B \cdot w_i k_i)$	$s_i \leftarrow s_i + (1-\alpha) \cdot K_i \cdot (\bar{g} - s_i)$
Per-agent variation	CANCELS OUT — see §4.1	SURVIVES via the kernel K_i
Key parameter	γ — sharp phase transition	λ — smooth relaxation rate
Convergence	Geometric to 0 or 1, ~12 rounds	To the weighted mean, then frozen
Who calls it	API, web app, SDK, all bridges	Its own CLI, nothing else

WHY A SECOND ALGORITHM WAS WRITTEN

The market simulator needed something Lineage A cannot provide: *graded* cooperation. A market maker doesn't binary-choose between quoting and not quoting; it quotes at a width. Lineage A's strategies are labels — A or B — so the market model needs a continuous variable in $[0,1]$, and once you need that, the argmax update no longer makes sense and you need a relaxation instead.

The second thing it needed was heterogeneity that persists. In a market, market makers, HFTs, institutions and retail should not converge at the same rate. Lineage B gets that from the exponential kernel over "distance from the centre of the market," which is a genuinely different mechanism from Lineage A's degree weighting.

So the fork is justified. What isn't justified is calling both of them BTUT with no note anywhere that they differ, and leaving the more sophisticated of the two unreachable from the product.

2.3 Where the code actually is

AREA	LINES	WHAT'S IN IT
btut_market_simulator/src	5,264	Kernel, order book, agents, metrics, config, CLI — the best-engineered code here
components/ + app/	~4,600	Nine Next.js routes and their React components
integrations/	~3,900	SUMO, drone swarm, ROS, plus validation experiments
lib/	~2,800	Engines, WASM loader, TraCI / ROS / AWS clients, React hooks
api/	~1,900	FastAPI service, middleware, error handling, collaboration
tests/ + benchmarks/	1,792	66 Python tests across unit, integration, e2e; two benchmark suites
rust-engine/src	1,142	Lineage A compiled to WebAssembly
17 root *.md files	~1,600	Overlapping deployment and status narratives, several contradicting each other

The core algorithm — the thing the entire platform exists to run — is about 260 lines. Everything else is delivery, integration, presentation, or the market simulator.

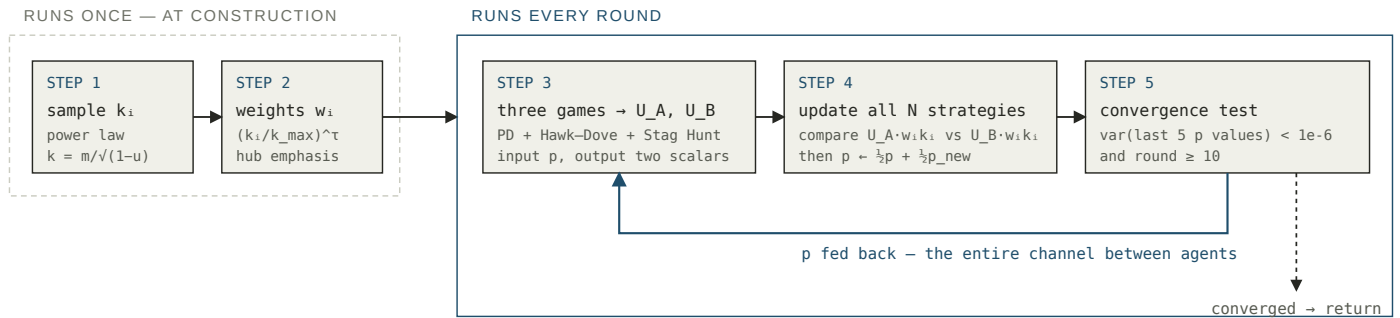
SECTION THREE

Engine A: the algorithm everything runs on

Five steps. Two run once at construction, three run every round.

Read in the order the code executes them.

Everything the platform shows a user comes out of this loop. It exists in Python, TypeScript and Rust; the Python is quoted below because the API runs it, with the others shown where they differ.



Cost per round: one $O(N)$ pass. No adjacency list is ever allocated — which is why the complexity is linear rather than quadratic.

Figure 2. Engine A's control flow. Steps 1 and 2 build per-agent state that never changes afterwards. Steps 3 to 5 loop. The feedback edge carrying p is what replaces the interaction graph.

3.1 Step 1 — drawing the degrees

Each agent gets a degree: a number standing in for how many neighbours it would have had if a graph existed. The graph never exists. The degree is drawn straight from the distribution a Barabási–Albert graph would have produced.

```
def _sample_degrees(self) -> np.ndarray:
    """Sample degrees from Barabási-Albert power-law distribution"""
    u = np.random.uniform(0, 1, size=self.N)
    degrees = self.m / np.sqrt(1 - u)
    return np.maximum(degrees, self.m)

def _compute_weights(self) -> np.ndarray:
    """Compute kernel weights:  $w_i = (k_i / k_{\max})^{\tau}$ """
    if self.tau == 0 or self.max_degree == 0:
        return np.ones(self.N)
    return (self.degrees / self.max_degree) ** self.tau
```

Both steps in one place, since they run back to back. The Rust and TypeScript versions `floor()` the degree to an integer where Python leaves it continuous — a small quantisation of the weights, not a behavioural difference.

3.2 Step 3 — three games, mixed in equal thirds

This is where the model's substance lives. Rather than picking one game, BTUT averages three, on the reasoning that real coordination problems have elements of all of them: the temptation to free-ride (Prisoner's Dilemma), the cost of mutual escalation (Hawk–Dove), and the payoff to matching expectations (Stag Hunt). Each takes the current cooperating fraction `p` and returns a payoff for choosing A and a payoff for choosing B.

```
def _compute_expected_utilities(self, p: float) -> Tuple[float, float]:
    """Compute expected utilities for both strategies"""

    # Base payoffs (log-scaled)
    u_A_PD = np.log(2.0)
    u_B_PD = np.log(1.2)
    u_A_HD = np.log(2.2)
    u_B_HD = np.log(1.5)
    u_A_SH = np.log(2.5)
    u_B_SH = np.log(1.2)

    # Prisoner's Dilemma
    EU_PD_A = 0.5 * (u_A_PD + (p * u_A_PD + (1 - p) * u_B_PD)) - self.cA_PD * u_A_PD
    EU_PD_B = 0.5 * (u_B_PD + (p * u_A_PD + (1 - p) * u_B_PD)) - self.cB_PD * u_B_PD

    # Hawk-Dove
    EU_HD_A = 0.5 * (u_A_HD + p * u_A_HD + (1 - p) * u_B_HD) * (p * self.alpha + (1 - p)) - 0.22 * u_A_HD
    EU_HD_B = 0.5 * (u_B_HD + p * u_A_HD + (1 - p) * u_B_HD) * (p + (1 - p) * 1.08) - 0.12 * u_B_HD

    # Stag Hunt
    EU_SH_A = 0.5 * (u_A_SH + p * u_A_SH + (1 - p) * u_B_SH) * (p * self.gamma + (1 - p) * 0.03) - self.cA_SH * u_A_SH
    EU_SH_B = 0.5 * (u_B_SH + p * u_A_SH + (1 - p) * u_B_SH) * (p * 0.03 + (1 - p)) - self.cB_SH * u_B_SH

    # Mixed game (equal weighting)
    mix = 1.0 / 3.0
    U_A = mix * (EU_PD_A + EU_HD_A + EU_SH_A)
    U_B = mix * (EU_PD_B + EU_HD_B + EU_SH_B)

    return U_A, U_B
```

Three things to notice. γ appears in exactly one of those six expressions — the Stag Hunt multiplier $(p \cdot \gamma + (1-p) \cdot 0.03)$. Tuning γ is tuning that one term. The Hawk–Dove costs, 0.22 and 0.12, are hard-coded where the other four costs are parameters. And every base payoff is log-scaled: $\ln(2.5)$ rather than 2.5 .

WHY LOG-SCALED PAYOFFS

The TypeScript port annotates this as "log-scaled for stability," and the reasoning holds. The three games' payoffs get multiplied by terms containing γ , which the API allows up to 10. Raw payoffs of 2.0 to 2.5 multiplied by a factor of 10 and then summed across three games produces numbers that swing hard as p moves; compressing to 0.18–0.92 via the logarithm keeps every intermediate in the same order of magnitude.

It also has the effect of preserving *ratios* rather than differences between the games, so the equal-thirds mixing weights mean something consistent regardless of how the raw payoffs are chosen.

The important structural fact about this stage: **whatever N is, it produces exactly two numbers.** Every agent in the population is about to be compared against the same pair. Section 4 is about what follows from that.

3.3 Step 4 — the update

Each agent scales both payoffs by its own weight and degree, adopts whichever is larger, and the new population fraction is blended with the old one at equal weight.

lib/simulation/btut_engine.py

lines 172–196, complete

```
def _update_strategies(self):
    """O(N) strategy update using mean-field approximation"""
    U_A, U_B = self._compute_expected_utilities(self.p)

    # Weighted voting for each agent
    weighted_A = 0.0
    total_weight = 0.0

    for i in range(self.N):
        agent_U_A = U_A * self.weights[i] * self.degrees[i]
        agent_U_B = U_B * self.weights[i] * self.degrees[i]

        if agent_U_A > agent_U_B:
            self.strategies[i] = 1
            weighted_A += self.weights[i]
        else:
            self.strategies[i] = 0

        total_weight += self.weights[i]

    p_new = weighted_A / total_weight if total_weight > 0 else self.p

    # Momentum update
    self.p = 0.5 * self.p + 0.5 * p_new
    self.history.append(self.p)
```

WHY THE MOMENTUM TERM

Without it, a population that all switches at once oscillates: everyone defects because p is low, which makes p low, which makes everyone defect. Best-response dynamics in games with strategic complements do this routinely.

Blending the new fraction with the old at $\frac{1}{2}$ damps the oscillation into a smooth approach. It is the same reason gradient descent uses a learning rate below one. Section 4.2 shows the damping is so effective that it fully determines the trajectory.

3.4 Where the three implementations disagree

Python and Rust do what you just read. TypeScript does something else: rather than adopting the better strategy outright, each agent computes a probability of switching from a logistic function of the payoff difference, and flips a coin.

lib/simulation/btut-engine.ts

lines 162–177 — a different rule

```
for (const agent of agents) {
  const weight = agent.weight;

  const current_u = agent.strategy === 'A' ? u_a : u_b;
  const other_u    = agent.strategy === 'A' ? u_b : u_a;

  const delta_u = other_u - current_u;
  const prob_switch = 1 / (1 + Math.exp(-gamma * weight * delta_u));

  if (this.rng() < prob_switch) {
    agent.strategy = agent.strategy === 'A' ? 'B' : 'A';
  }

  agent.utility = current_u;
}
```

	PYTHON AND RUST/WASM	TYPESCRIPT
Decision rule	Deterministic — take the better one	Logistic switching probability
Role of γ	Payoffs only	Payoffs, <i>and</i> as the logistic steepness
Momentum on p	Yes	No — p is recounted directly
Can agents differ?	NO	YES — weight survives inside the exponent
Reached when	API call, or browser with WASM	Browser where WASM failed to load

This is a real behavioural fork, not an optimisation. The same configuration produces a qualitatively different simulation depending on whether a visitor's browser loaded a WebAssembly module. The simulator page does display which engine is live, so the difference is visible — but it is never described as a difference in the model.

3.5 Step 5 — stopping

lib/simulation/btut_engine.py

lines 214–226

```
        if state['converged'] and iteration >= 10:
            break

    return states

def _check_convergence(self, window: int = 5, threshold: float = 1e-6) -> bool:
    """Check if simulation has converged"""
    if len(self.history) < window:
        return False

    recent = self.history[-window:]
    variance = np.var(recent)
    return bool(variance < threshold)
```

A variance test over the last five values of p , with a floor of ten rounds before early exit is allowed. The TypeScript engine uses the same test with a window of **100** samples rather than 5 — so with the 100-iteration `quick` preset it can only ever evaluate convergence on the final step.

SECTION FOUR

What Engine A does when you run it

Reading code tells you what it computes. Running it tells you what it does. This engine has a closed-form solution its documentation doesn't mention.

4.1 The per-agent factor cancels

Return to the two lines the deterministic update turns on:

lib/simulation/btut_engine.py

lines 181–184

```
agent_U_A = U_A * self.weights[i] * self.degrees[i]
agent_U_B = U_B * self.weights[i] * self.degrees[i]

if agent_U_A > agent_U_B:
```

`U_A` and `U_B` are the same two scalars for every agent — Step 3 produced exactly two numbers. The factor `weights[i] * degrees[i]` is strictly positive: weight is $(k/k_{\max})^\tau$ which lies in $(0, 1]$, and degree is at least $m \geq 1$. Multiplying both sides of a comparison by the same positive number cannot change which side is larger. So the comparison reduces, for every agent, to `U_A > U_B`.

AS WRITTEN — N INDEPENDENT COMPARISONS

```
agent 0: U_A · w_0 k_0 > U_B · w_0 k_0 ?
agent 1: U_A · w_1 k_1 > U_B · w_1 k_1 ?
⋮
agent N-1: U_A · w_N k_N > U_B · w_N k_N ?
```

divide by
 $w_i k_i > 0$

WHAT IT EVALUATES TO

```
every agent: U_A > U_B ?
one scalar comparison, evaluated N times
⇒ all N agents adopt the same strategy
⇒ p_new is exactly 0 or exactly 1, never between
```

SO THE OUTER LOOP BECOMES

```
p ← ½ · p + ½ · p_new with p_new ∈ {0, 1} ⇒ p halves its distance to 0 or 1 every single round
```

Figure 3. Why the N-agent loop is a scalar recurrence in disguise. The weight and degree are computed, stored, and multiplied in — then divided straight back out by the comparison operator. The identical cancellation is present in `rust-engine/src/lib.rs` lines 239–246.

4.2 Measured: every agent agrees, every round

That gives a testable prediction — at any round, the number of distinct strategy values across the whole population should be exactly one. Importing the shipped engine unmodified, stepping it, and counting distinct values after each step:

measured — $N = 1000$, $\gamma = 1.45$, $\tau = 0.30$

verification run

```
round 1: p=0.750000 distinct strategies among 1000 agents = [1]
round 2: p=0.875000 distinct strategies among 1000 agents = [1]
round 3: p=0.937500 distinct strategies among 1000 agents = [1]
round 4: p=0.968750 distinct strategies among 1000 agents = [1]
round 5: p=0.984375 distinct strategies among 1000 agents = [1]
round 6: p=0.992188 distinct strategies among 1000 agents = [1]
```

And the p values are not arbitrary. With $p_0 = 0.5$ and p_{new} pinned at 1, the recurrence

$p \leftarrow \frac{1}{2}p + \frac{1}{2}$ unrolls exactly:

$$p_t = 1 - 2^{-(t+1)} \quad (\text{cooperating branch}) \quad p_t = 2^{-(t+1)} \quad (\text{defecting branch})$$

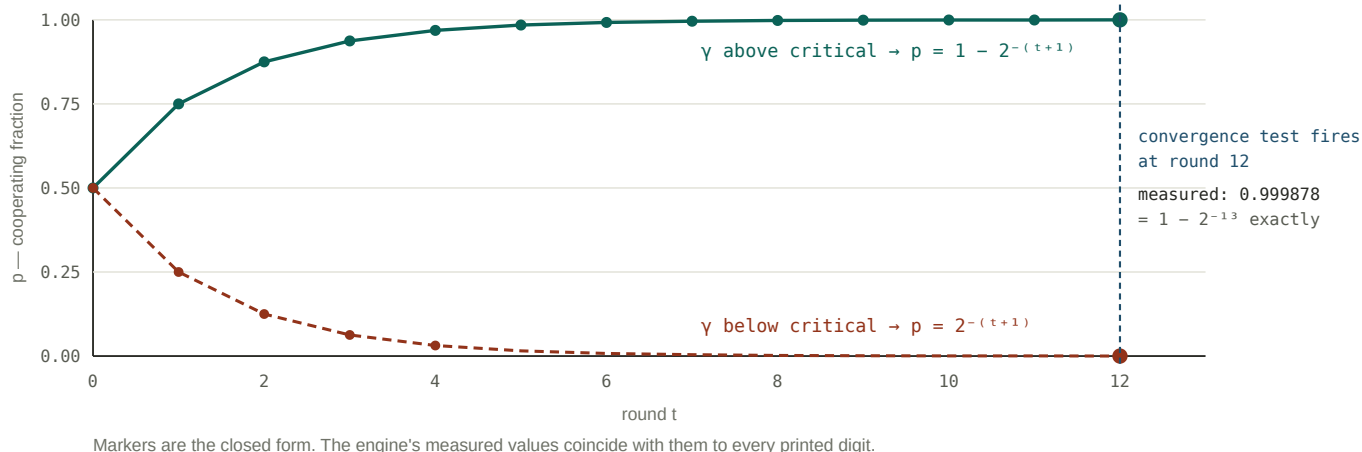


Figure 4. The trajectory is a halving of the gap to the boundary, and nothing else. The measured terminal value 0.999878 equals $1 - 2^{-13}$ to six decimal places; the defecting branch's 0.000122 equals 2^{-13} likewise.

4.3 Measured: the answer doesn't depend on N

If the trajectory is a scalar recurrence, N should affect run time and nothing else.

Measured across three orders of magnitude on the unmodified engine, with a 20-round cap:

N	ROUNDS RUN	WALL TIME	MS PER AGENT-ROUND	FINAL P
1,000	12	0.024 s	2.033	0.999878
10,000	12	0.094 s	0.787	0.999878
100,000	12	0.914 s	0.761	0.999878

Two readings, both true. The favourable one: wall time is genuinely linear in N — ten times the agents costs 9.7 times the time, and per-agent cost is flat once the interpreter is warm. The $O(N)$ claim is empirically supported, and that was the headline claim. The other reading: the final value is bit-identical across a hundredfold change in population, because the population never enters the answer.

4.4 Measured: the phase transition is a step

Γ	1.30	1.40	1.44	1.45	1.50	1.60
final p	0.000122	0.999878	0.999878	0.999878	0.999878	0.999878

$N = 1000$, $\tau = 0.30$, 25-round cap. The transition sits between 1.30 and 1.40, and it is a hard step — there is no γ at which the population partially cooperates. This is what Section 7's calibrator is built to find, and it explains why that search converges so cleanly: it is bisecting a step function, the easiest possible target.

4.5 The ledger

CLAIM	STATUS	EVIDENCE
Runs in $O(N)$ time	HOLDS	Flat per-agent cost, §4.3
Memory is $O(N)$; no graph stored	HOLDS	Three flat arrays; the <code>neighbors</code> field is declared and never populated
Scales to a million agents	HOLDS	Extrapolates to ~ 9 s in Python; the Rust kernel does 100k in 2.5 ms
Sharp cooperation phase transition in γ	HOLDS	Measured, §4.4
Converges to a stable equilibrium	HOLDS	Provably — the recurrence is a contraction with ratio $\frac{1}{2}$
Agents coordinate <i>with each other</i>	DOES NOT HOLD	All agents share one decision, §4.1–4.2
τ (hub influence) changes the outcome	DOES NOT HOLD	τ enters only via w_i , which cancels. It <i>does</i> matter in the TypeScript engine and both domain bridges.

WHY THIS HAPPENED, AND THE ONE-LINE REPAIR

The design instinct was right. Scaling each agent's payoff by its influence is exactly what you'd want in a heterogeneous population, and it's what makes the model more than a single differential equation. The slip is that the scaling was applied *symmetrically* — to both sides of a comparison — and a comparison is invariant to positive scaling. Had the same factor been applied to an additive term, or to only one side, or inside a nonlinearity, it would have survived.

Which is precisely what the TypeScript port does: `1 / (1 + exp(- $\gamma \cdot w_i \cdot \Delta u$ ))` puts the weight inside an exponential, where scaling changes the result. That formulation is already in this repository, already written, already working. Adopting it in `_update_strategies` restores per-agent heterogeneity, keeps the $O(N)$ property, keeps the phase transition, and changes no public interface.

Engine B: the market kernel

A second algorithm, sharing only vocabulary with the first. It is the better-engineered of the two, and nothing in the product can reach it.

Inside `btut_market_simulator` is a kernel with no games, no strategy labels, and no phase transition. Cooperation is a continuous number in $[0,1]$ per agent, and each round every agent moves a fraction of the way toward a weighted population average. How large that fraction is depends on an exponential kernel over the agent's "distance from the centre of the market."

It is also the most carefully written code in the repository: structure-of-arrays memory layout, optional Rayon parallelism behind a feature flag, tracing instrumentation on the hot paths, and a scratch buffer that gets swapped rather than reallocated.

5.1 How an agent becomes two numbers

Two derived quantities give agents their individuality. *Weight* decides how much an agent's opinion counts toward the average. *Distance* decides how strongly that average pulls back on the agent.

```

fn compute_weight(agent: &AgentState) -> f64 {
    let type_weight = match agent.agent_type {
        AgentType::Retail => 1.0,
        AgentType::HFT => 5.0,
        AgentType::Institution => 20.0,
        AgentType::MarketMaker => 10.0,
    };
    // Weight scales with sqrt of capital (diminishing influence)
    type_weight * agent.capital.sqrt().max(1.0)
}

fn compute_distance(agent: &AgentState) -> f64 {
    let type_distance = match agent.agent_type {
        AgentType::MarketMaker => 0.1, // Closest to the market
        AgentType::HFT => 0.3,
        AgentType::Institution => 0.6,
        AgentType::Retail => 1.0, // Farthest from core
    };

    let capital_factor = 1.0 / (1.0 + (agent.capital / 10000.0).ln().max(0.0));
    type_distance * capital_factor
}

```

TRADER CLASS	WEIGHT	DISTANCE	KERNEL K	READING
Market maker	10.0	0.1	0.990	Sits on the book — adapts fastest to consensus
HFT	5.0	0.3	0.970	Near the book, fast, small size
Institution	20.0	0.6	0.942	Heaviest vote, slowest to move
Retail	1.0	1.0	0.905	Lightest vote, least responsive

WHY SQUARE ROOT OF CAPITAL, AND WHY THE CLASSES ARE ORDERED THIS WAY

The square root is a deliberate choice of diminishing returns. A fund with a hundred times the capital of another should have more influence on the market's consensus — but not a hundred times more, or the model becomes a single fund and some noise. Square-root scaling gives it ten times. This is the same instinct behind the square-root law of market impact in real execution research.

The distance ordering encodes a genuine market intuition. A market maker's entire business is reading and reacting to consensus, so it should adapt almost instantly ($K = 0.99$). A retail trader looks at their portfolio once a week ($K = 0.905$). Institutions get the heaviest weight but the slower kernel: they move the market when they act, and they act slowly.

Note the spread of K values is narrow at the default decay $\lambda = 0.1$ — from 0.905 to 0.990, less than 10%. The class distinctions are designed in but barely bite until λ is raised. That looks like an untuned default rather than a decision.

5.2 The update

btut_market_simulator/src/btut/kernel.rs

lines 241–314, parallel branch

```
fn update_iteration(&mut self) -> f64 {
    // Step 1: Compute weighted mean field (global cooperation level)
    let total_weight: f64 = self.weights.sum();
    let weighted_sum: f64 = Zip::from(&self.strategies)
        .and(&self.weights)
        .fold(0.0, |acc, &s, &w| acc + s * w);

    let global_cooperation = if total_weight > 0.0 {
        weighted_sum / total_weight
    } else {
        0.5
    };

    // Step 2: Update each agent's strategy
    self.new_strategies
        .as_slice_mut()
        .unwrap()
        .par_iter_mut()
        .enumerate()
        .for_each(|(i, new_s)| {
            let old_s = strategies[i];
            let k = kernel_values[i];

            // Target strategy based on mean field + bias
            let target = (global_cooperation + config.bias).clamp(0.0, 1.0);

            // Softmax-style update with temperature
            let influence = k * (target - old_s) / config.temperature;

            // Momentum-based update
            *new_s = config.momentum * old_s + (1.0 - config.momentum) * (old_s + influence);
            *new_s = new_s.clamp(0.0, 1.0);
        });

    // Step 3: Compute max delta and swap strategies
    let max_delta = Zip::from(&self.strategies)
        .and(&self.new_strategies)
        .fold(0.0f64, |acc, &old, &new| acc.max((new - old).abs()));

    std::mem::swap(&mut self.strategies, &mut self.new_strategies);

    max_delta
}
```

The momentum line looks like a blend of two candidate states, but expanding it collapses to something simpler:

algebraic identity, verified numerically

$$\alpha \cdot s + (1-\alpha) \cdot (s + i) = \alpha \cdot s + (1-\alpha) \cdot s + (1-\alpha) \cdot i = s + (1-\alpha) \cdot i$$

so: $s_i \leftarrow s_i + \underbrace{(1 - \text{momentum})}_{0.30 \text{ by default}} \cdot K_i \cdot (\text{mean} + \text{bias} - s_i) / \text{temperature}$

Momentum is not blending states; it is a step size on the correction. At the default 0.7, each agent closes $30\% \times K_i$ of its gap per round. Checked numerically: both forms give 0.459000000000 for $\alpha = 0.7$, $s = 0.42$, $i = 0.13$.

Unlike Lineage A, here the per-agent term genuinely survives — K_i scales a *step size*, not both sides of a comparison, so agents with different kernels really do converge at different rates. This is the mechanism Lineage A was reaching for and missed.

WHY STRUCTURE-OF-ARRAYS INSTEAD OF A VECTOR OF AGENT STRUCTS

The hot loop touches only two of the five per-agent quantities: `strategies` and `kernel_values`. In a vector-of-structs layout, walking a million agents would pull the weight, distance and scratch fields into cache alongside them and waste most of every cache line.

Five parallel flat arrays means the loop streams two contiguous runs of `f64` — SIMD-friendly and cache-dense. It also makes the parallel version nearly free: `par_iter_mut()` on a slice splits it into disjoint ranges with no copying and no synchronisation. And the double buffer means the update never allocates: `mem::swap` exchanges two pointers.

The price is that array index is no longer the agent's identity. The kernel's arrays and the simulation's `AgentState` vector must stay in the same order forever, or `sync_to_agents` silently writes each agent's cooperation into a different agent. Nothing in the type system enforces that.

5.3 Shocks, and why three of the four are permanent

SHOCK	EFFECT	WHO IT HITS	WHAT HAPPENS AFTER
Liquidity crisis	$s - 0.5 \cdot \text{severity}$	Everyone, uniformly	Moves the mean itself — new lower equilibrium, permanent
Flash crash	$s \times 0.3$	Only agents with distance < 0.5 — market makers and HFT	Moves the mean partly — partial permanent drop
Regeneration	$s + 0.3 \cdot \text{strength}$	Everyone, uniformly	The inverse of a liquidity crisis
Noise	$s + N(0, \sigma)$	Independent draw per agent	Mean roughly preserved — the one shock that is recovered from

The kernel's only restoring force is the population mean, so any shock that moves the mean moves the equilibrium with it. Only the noise shock, which perturbs agents in both directions and leaves the mean intact, gets relaxed away. That is a real property of the design rather than a bug, but it means "recovery from a liquidity crisis" is not something this model can exhibit without an external target. The `FlashCrash` variant's `duration` field is destructured to `_` and never read — the crash is instantaneous whatever value you pass.

SECTION SIX

The market simulator

The kernel is one stage of a six-stage loop that generates orders, matches them in a real limit order book, and scores the result for crashes.

Simulation::step() — simulation.rs lines 283–320

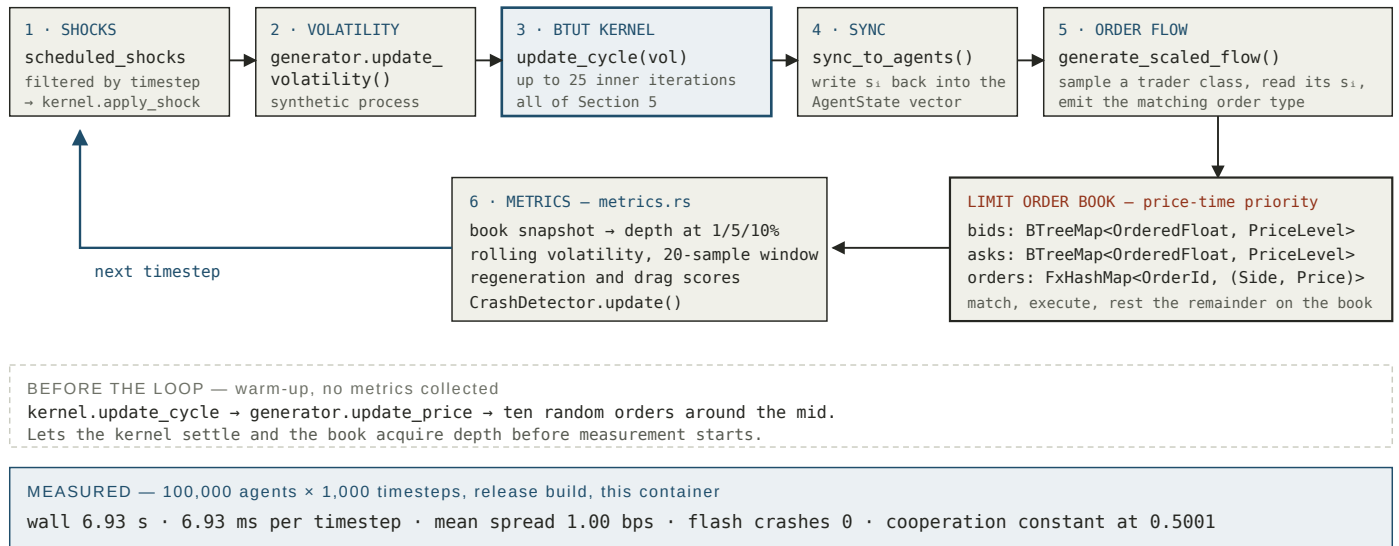


Figure 5. The market timestep. Stage 3 is the whole of Section 5; stages 5 and 6 are the market machinery around it. The bottom strip is a real run of this exact loop.

6.1 How cooperation becomes an order

This is the coupling that makes it a market model rather than an abstract one. An agent's cooperation level decides what *kind* of order it sends. Cooperators post passive limit orders and two-sided quotes — they supply liquidity. Defectors send aggressive market orders — they consume it.

AGENT CONDITION	ORDER EMITTED	EFFECT ON THE BOOK
Cooperative and a liquidity provider market makers, HFT with $s > 0.5$	Two-sided quote at $\text{mid} \pm h$, where $h = \text{spread} \times (0.3 + 0.7 \cdot (1 - \text{mean}))$	Adds depth on both sides. More cooperation $\rightarrow$ tighter quotes.
Cooperative, not a provider	Passive limit order, one side, offset $\text{spread} \times U(0.5, 2.0)$	Adds depth on one side
Defecting	Market order at half size — crosses the spread	Consumes depth. This is how low cooperation widens spreads and trips crash detection.

Notice the quote-width term in the first row: it reads `mean_coop`, which the kernel produced, and sets the spread, which becomes the next timestep's offsets. Cooperation is genuinely in the loop here, not decorative.

6.2 The one approximation, stated honestly

The order-flow stage does not simulate every agent. It computes how many orders the population *would* produce, then caps the number actually generated and inflates their sizes to compensate.

btut_market_simulator/src/simulation.rs

lines 380–401

```

/// Generate scaled order flow based on agent population statistics
///
/// This is an O(1) approximation of agent behavior that scales to millions of agents.
fn generate_scaled_flow(&mut self) {
    let num_agents = self.config.simulation.num_agents;
    ...
    let total_activity = retail_activity + hft_activity + inst_activity + mm_activity;
    let expected_orders = (num_agents as f64 * total_activity) as usize;

    // Scale down to reasonable number while preserving statistics
    let actual_orders = expected_orders.min(1000);
    let order_scale = expected_orders as f64 / actual_orders as f64;

```

WHY CAP THE ORDER FLOW, AND WHAT IT COSTS

A million agents at market-maker activity rates would generate hundreds of thousands of orders per timestep. Each one walks the book, allocates a trade record, and mutates two BTreeMaps. That is not a hundred times slower than a thousand orders — it is a hundred times slower *and* the book grows without bound, so every subsequent match walks a longer structure. The simulation would not finish.

Capping at a thousand orders with proportionally larger sizes preserves the first moment of order flow — total volume is right — at the cost of the distribution. A thousand orders of size 100 do not hit a book the way a hundred thousand orders of size 1 do: the price impact is lumpier, the queue dynamics are wrong, and rare events involving many small orders can't occur.

The honest summary is that the kernel is genuinely $O(N)$ in agents, while the market it feeds is a fixed-resolution sample of that population. "One million agents" describes the kernel's workload accurately and the order book's inaccurately. The code says so in its own doc comment, which is more than most such approximations get.

6.3 The matching engine

The book is two BTreeMap s keyed by price — bids read in reverse for best-first, asks read forward. Each price level holds a SmallVec<[Order; 8]>, an inline array that only touches the heap past eight resting orders at one price. Price priority comes from the map ordering, time priority from FIFO position within the level.

```

fn match_bid(&mut self, order: &mut Order, trades: &mut Vec<Trade>) {
    // Collect prices to process (ascending order - best ask first)
    let prices_to_check: Vec<Price> = self.asks.keys().cloned().collect();

    for ask_price in prices_to_check {
        // For limit orders, stop if we've passed the limit price
        if order.order_type == OrderType::Limit && ask_price > order.price {
            break;
        }
        if order.remaining <= 0.0 { break; }

        let trade_price = ask_price.0;

        if let Some(level) = self.asks.get_mut(&ask_price) {
            let mut i = 0;
            while i < level.orders.len() && order.remaining > 0.0 {
                let passive = &mut level.orders[i];
                let fill_qty = order.remaining.min(passive.remaining);

                if fill_qty > 0.0 {
                    let trade = Trade { /* aggressor, passive, price, quantity */ };
                    order.remaining -= fill_qty;
                    passive.remaining -= fill_qty;
                    level.total_quantity -= fill_qty;
                    trades.push(trade);
                }

                if passive.remaining <= 0.0 { level.orders.remove(i); } else { i += 1; }
            }
        }
        self.asks.retain(|_, level| !level.is_empty());
    }
}

```

The trade price is always the *passive* order's price — the resting order sets the terms. That is correct for a continuous double auction, and it is the detail most toy order books get wrong. Seven unit tests cover this path and all pass.

6.4 What a real run does

WALL CLOCK	PER TIMESTEP	KERNEL, 100K AGENTS
6.93 s	6.93 ms	2.49 ms
MEAN SPREAD	FLASH CRASHES	COOPERATION
1.00 bps	0	0.5001
MIN COOPERATION	REALISED VOLATILITY	
0.5001	0.0000	

The simulator's own summary printer describes the cooperation series as *"Constant value 0.5001."* Everything is frozen. The reason is the fixed point from Section 5.2: agents start near 0.5, so the weighted mean is 0.5, so every agent's target equals its current value, so the correction is zero. The kernel reports convergence in a single iteration for 999 of the 1,000 timesteps, and spread and volatility flatline in sympathy because order flow is driven by cooperation.

WHY THE DEFAULT CONFIGURATION SITS EXACTLY ON THE FIXED POINT — AND HOW TO MOVE IT

The machinery is sound; the defaults are on the one configuration where nothing happens. `cooperation_bias` is 0, so the target is the current mean. `cooperation_noise` is low, so agents start together and there's no gap to close. And no shock is scheduled.

Three levers move it off: set a non-zero bias so the target differs from the mean; raise the initial noise so agents start dispersed and the kernel has work to do; or schedule shocks, which the CLI already exposes as `--shock-at`. The `shock` preset does the third — it is the one preset configured to produce a moving series, which strongly suggests the defaults were never run and looked at.

SECTION SEVEN

Calibration: choosing gamma

Because the outcome is a step function in γ , picking γ is the configuration problem. This is the machinery built to pick it automatically.

Section 4.4 measured it: below the critical γ the population collapses to universal defection, above it saturates at universal cooperation, and there is no middle. Getting γ wrong doesn't degrade the result gracefully — it inverts it. The critical point finder's own docstring calls this "THE key systematic step that must run before any simulation," which is the right emphasis.

The algorithm is a bisection. Run the engine at the bottom of a bracket, run it at the top, confirm they land on opposite sides, then repeatedly halve.

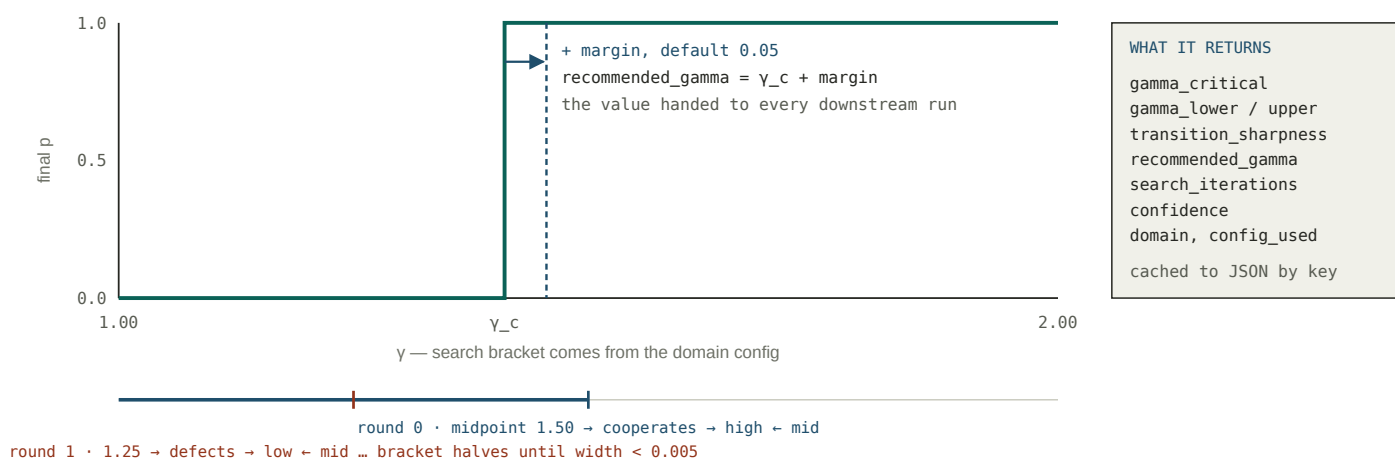


Figure 6. Bisection against the step function measured in Section 4.4. Because the target really is a step, the search is unusually well-conditioned — twenty rounds at tolerance 0.005 is far more resolution than the problem needs.

```
# Verify bounds
coop_low = self._run_simulation(low, tau, N)
coop_high = self._run_simulation(high, tau, N)

if coop_low > 0.5:
    logger.warning(f"Lower bound gamma={low} already shows cooperation. Extending search.")
    low = max(0.5, low - 0.5)
    coop_low = self._run_simulation(low, tau, N)

if coop_high < 0.5:
    logger.warning(f"Upper bound gamma={high} still shows defection. Extending search.")
    high = high + 0.5
    coop_high = self._run_simulation(high, tau, N)

# Binary search
while high - low > tolerance and iterations < max_iterations:
    mid = (low + high) / 2
    coop_mid = self._run_simulation(mid, tau, N)

    if coop_mid < 0.5:
        low = mid
    else:
        high = mid

    iterations += 1

gamma_critical = (low + high) / 2

# Validate transition sharpness
test_gammas = np.linspace(gamma_critical - 0.05, gamma_critical + 0.05, 11)
test_coops = [self._run_simulation(g, tau, N) for g in test_gammas]

transition_width = 0.0
for i in range(len(test_coops) - 1):
    if test_coops[i] < 0.1 and test_coops[i+1] > 0.9:
        transition_width = test_gammas[i+1] - test_gammas[i]
        break

sharpness = 1.0 - min(1.0, transition_width / 0.1)
recommended_gamma = gamma_critical + margin
```


WHY THE BOUNDS CHECK, AND WHY THE SAFETY MARGIN

The bounds check is the best part of this design. Bisection silently returns garbage if both endpoints are on the same side of the boundary — it just converges to whichever endpoint, with no error. Testing both ends first, and widening once in each direction when they agree, converts a silent wrong answer into a warning and a retry.

The margin exists because γ_c is estimated from finite simulations with a random seed, so the true boundary is somewhere in a small interval around it. Operating *at* γ_c means operating in the bistable regime where the outcome depends on initial conditions and could flip between runs. Sitting 0.05 above buys determinism. The cost is that you are no longer at the cheapest configuration that achieves cooperation — which for a real deployment is a sensible price.

Four domains ship with pre-tuned brackets and costs. Note that `traffic`'s costs are identical to `abstract` — only N and the search bracket differ — while `drone` and `robot` carry genuinely distinct cost structures.

DOMAIN	N	Γ BRACKET	CA_SH	CB_SH	CA_PD	CB_PD	A
abstract	1,000	1.0 – 2.0	0.40	0.10	0.20	0.08	0.60
traffic	500	1.2 – 1.8	0.40	0.10	0.20	0.08	0.60
drone	100	1.2 – 2.0	0.35	0.12	0.18	0.10	0.50
robot	50	1.1 – 1.8	0.45	0.08	0.22	0.06	0.70

Results are cached to JSON keyed by `{domain}_tau{ $\tau$ }_N{ $N$ }`, so a repeat calibration is free. The cache path is *relative* — it lands wherever the process working directory happens to be, which on a read-only serverless filesystem is where the write fails.

SECTION EIGHT

The service layer

834 lines of FastAPI exposing the Python engine — single runs, batches, sweeps, benchmarks, calibration, and a WebSocket.

ROUTE	RATE LIMIT	WHAT IT DOES
GET /health	—	Status, running count, uptime
GET /api/presets	—	Six named configurations
POST /api/simulate	100/min	One run. <code>async_mode</code> chooses inline or background execution
GET · DELETE /api/simulate/{id}	—	Fetch or drop a stored result
POST /api/simulate/batch	10/min	Up to 100 configs, gathered concurrently or run in sequence
POST /api/simulate/sweep	10/min	Vary one of γ , τ , N , α across 2–50 values
POST /api/benchmark	5/min	Timing across agent counts, plus a hard-coded comparison table
POST /api/calibrate	10/min	Critical point discovery for one domain
GET /api/calibrate/recommended/{domain}	—	Just the γ number
POST /api/calibrate/all	5/min	abstract, traffic and drone in one call
GET /api/stats	—	Aggregate counts and mean runtime
POST · GET /api/projects	100/min	Grouping container; owner hard-coded to <code>"anonymous"</code>
WS /ws	—	Broadcasts completion and failure events

The executor is straightforward: mark the record running, build the simulator, call `run()`, extract the final state and the convergence history, mark it complete, and broadcast on the socket. Errors are caught, recorded as `failed` with the message, and broadcast too — so a failing simulation doesn't take down a request.

WHY THE THREE SHORTCUTS HERE ARE WORTH KNOWING ABOUT

The blocking call. `simulator.run()` is synchronous CPU work — measured at 0.91 s for $N = 100,000$, and the API accepts N up to ten million. It is awaited inside a coroutine with no thread-pool hand-off, so it holds the event loop for its entire duration and stalls every other request on that worker. The fix is `await asyncio.to_thread(simulator.run)`, one line. This is the single highest-impact defect in the service.

The in-memory stores. `simulations_db`, `projects_db` and `rate_limit_db` are module-level dictionaries. Not shared across the four uvicorn workers the Dockerfile starts, not persisted across restarts. A client can POST a simulation, get an ID back, and receive a 404 on the follow-up GET because the request landed on a different worker. The code comments say "Use Redis/PostgreSQL in production" and the compose file already provisions Redis — the wiring was just never done.

The rate limiter follows the same fate. Per-worker dictionaries mean the effective limit is four times the configured one, and it resets on deploy.

All three are the same shape of decision: a correct single-process implementation shipped with a note describing the multi-process fix, and the fix not yet applied. That's a reasonable way to move fast; it stops being reasonable once the thing is deployed behind `--workers 4`, which it is.

The client surfaces

Two web applications, two browser engines, and an SDK that doesn't contain the algorithm.

9.1 The web application

Nine Next.js routes. The landing page is results-first — stress test, strategy comparison, τ sweep, an embedded live simulator, a scaling proof. `/simulator` is the full control surface with presets, live metrics, a network view and JSON export. `/traffic` and `/robotics` are control panels for the domain bridges and need a running TraCI or rosbridge to do anything. `/docs` is 565 lines of mathematical exposition.

The notable architectural decision is that simulations run *in the browser*, not on a server.

WHY RUN THE ENGINE CLIENT-SIDE

A public demo that computes server-side needs capacity planning, a queue, per-visitor rate limits, and a cold-start story — and every visitor costs money. A demo that computes in the visitor's own browser has none of those. It also means the interactive simulator can animate at frame rate, stepping and redrawing continuously, which a network round trip per step could never do.

The cost is that you now need the algorithm compiled for the browser, and a fallback for when that fails — which is where the third and fourth implementations came from, and where the model divergence of Section 3.4 crept in.

```
export class BTUTUnifiedSimulator {
  private simulator: TypeScriptSimulator | BTUTWasmSimulator;
  private usingWasm: boolean;

  constructor(config: SimulationConfig) {
    this.usingWasm = isWasmAvailable();

    if (this.usingWasm) {
      console.log('Using WASM implementation');
      this.simulator = new BTUTWasmSimulator(config);
    } else {
      console.log('Using TypeScript implementation');
      this.simulator = new TypeScriptSimulator(config);
    }
  }
}
```

Both are wrapped behind one `step()` / `run()` / `reset()` interface, so callers can't tell which model they got. The page does surface a badge — "Rust/WASM Accelerated" or "TypeScript Engine" — but as a performance note, not a model note.

9.2 There are two web applications

The root `app/` tree is the nine-route application described above, and it is what `npm run build` compiles. A second, smaller Next.js app lives in `frontend/src/app/` with its own package.json, tsconfig and lockfile — a single landing page with Hero, StatCard, PriceChart, CooperationChart and AgentDistribution components, built for the market simulator of Sections 5 and 6.

`vercel.json` routes production traffic to the second one, while `.vercelignore` excludes the Python API that the same file lists as a build target. Section 12 has the details.

9.3 The Python SDK is a client

Worth stating plainly, because the README's quick-start implies otherwise: `python-sdk/btut` imports `requests` and posts to the API. It contains no copy of the algorithm. `pip install btut-sdk` followed by `Simulator(agents=100000).run()` needs a reachable API host. The ROS node and the AWS Lambda handler both import this SDK, so both inherit the same dependency.

SECTION TEN

The domain bridges

Traffic, drones and robots. This is where the mean-field model meets physical space — and where, quietly, it becomes a better model.

The abstract engine has no geometry. Real vehicles and drones do. The bridges solve this by computing *degree* from actual spatial proximity: a vehicle's degree is the number of other vehicles within a 50-metre communication radius; a drone's is the number of drones within its 3-D communication range. Hubs are no longer a statistical fiction — they are the cars sitting in the middle of a busy intersection.

integrations/sumo/btut_sumo_bridge.py

strategy update, spatial variant

```
for vid, vehicle in vehicles.items():
    w = weights.get(vid, 0.01)
    k = vehicle.degree

    # Weighted utility comparison
    agent_U_A = EU_A * w * (k + 1) # +1 to avoid zero
    agent_U_B = EU_B * w * (k + 1)

    if agent_U_A > agent_U_B:
        strategies[vid] = "A"
        weighted_A += w
    else:
        strategies[vid] = "B"

    total_weight += w

new_p = weighted_A / total_weight if total_weight > 0 else 0.5
smoothed_p = 0.5 * p_current + 0.5 * new_p
```

WHY THE BRIDGES ARE MORE THAN A WRAPPER — AND WHERE THEY INHERIT THE SAME FLAW

The genuine contribution here is that `degree` becomes a measured quantity. In the abstract engine it is drawn from a distribution and is pure fiction; in the SUMO bridge it is recomputed every step from real positions, so it changes as traffic flows, congestion forms, and vehicles cluster at intersections. The weights $(k/k_{\max})^\tau$ then vary over time in a way that tracks the actual road network. That is a real model, and it is what the validation experiments in `integrations/sumo/` are exercising.

But look at the comparison: $EU_A \cdot w \cdot (k+1)$ against $EU_B \cdot w \cdot (k+1)$. The same positive factor on both sides. The spatial bridge reproduces the cancellation of Section 4.1 exactly — the carefully measured degrees are computed, multiplied in, and divided back out. The drone swarm core has the same shape.

This is the strongest argument for fixing the update rule. The bridges did the hard part: they made degree mean something physical. One change to how that number enters the decision would make all three domains genuinely spatial.

SECTION ELEVEN

Deployment and verification

Four deployment targets, and what is actually tested.

TARGET	CONFIG	WHAT IT RUNS
Docker Compose	<code>docker-compose.yml</code>	Backend, frontend, Redis, nginx — all four with health checks. The most complete definition.
Fly.io	<code>fly.toml</code>	API only. 2 GB / 2 CPU, autostop, HTTP health check on <code>/health</code> .
Vercel	<code>vercel.json</code>	The <code>frontend/</code> app plus the Python API as a serverless function — see Section 12.
AWS Lambda	<code>lambda/handler.py</code>	A thin handler that parses the event body and calls the SDK.

On verification: the Rust market simulator is well covered. **36 tests, all passing** on this checkout — kernel initialisation, convergence, shock application, cooperation by type, large scale, seven order-book tests including matching and cancellation, crash detection, the regeneration score, CSV export, RNG reproducibility and forking, the synthetic generator, config validation and TOML round-tripping, plus four simulation-level tests including the baseline comparison.

The Python side has 66 test functions across unit, integration and end-to-end files. What no test covers is the thing that matters most: `nothing asserts that the Python, TypeScript and Rust engines produce the same output for the same seed.` That single test would have caught the divergence in Section 3.4 the day it appeared.

SECTION TWELVE

Everything that is wrong or unfinished

The complete list, with evidence, ordered by consequence.

	FINDING	WHERE	CONSEQUENCE
1	The per-agent factor cancels from both sides of the strategy comparison, so all N agents make one identical decision	<code>btut_engine.py:181</code> <code>rust-engine/lib.rs:239</code>	Heterogeneity, τ , and the degree distribution have no effect on the deterministic engines. Measured in §4.2.
2	The same cancellation is reproduced in the SUMO and drone bridges	<code>btut_sumo_bridge.py</code> <code>btut_swarm_core.py</code>	Carefully measured spatial degrees are computed and then have no influence.
3	TypeScript uses logistic switching where Python and Rust use argmax; TypeScript also has no momentum	<code>btut-engine.ts:154–178</code>	The browser silently runs a different model when WASM fails to load.
4	Blocking CPU work awaited on the event loop	<code>api/main.py:227</code>	One large simulation stalls every concurrent request on that worker.
5	All state is in module-level dicts	<code>api/main.py:63–65</code>	Results are invisible across workers and lost on restart; rate limits are per-worker.
6	Market kernel defaults sit exactly on the fixed point	<code>config.rs defaults</code>	A 1,000-timestep run produces a completely static market. Measured in §6.4.
7	<code>.vercelignore</code> excludes <code>api/</code> and <code>**/*.py</code> , which <code>vercel.json</code> lists as a build source	<code>.vercelignore</code> <code>vercel.json</code>	The serverless API cannot be built from that configuration.
8	Two Next.js applications; the root one is built by npm, the <code>frontend/</code> one is routed by Vercel	<code>package.json</code> <code>vercel.json</code>	Local development and production serve different applications.
9	The market simulator is unreachable from any product surface	<code>btut_market_simulator/</code>	5,264 lines of the best code here, available only via its own CLI.
10	Order flow is capped at 1,000 orders per timestep	<code>simulation.rs:399</code>	Past ~3,000 agents the order book stops scaling with N. Documented in-code.
11	No cross-implementation equivalence test	<code>tests/</code>	The three engines can drift apart indefinitely without failing anything.
12	Calibration cache writes to a relative path	<code>critical_point_finder.py:383</code>	Fails on read-only or ephemeral filesystems; lands unpredictably otherwise.
13	Dead and unused code: <code>kernelWeight()</code> never called (and uses the opposite exponent sign), <code>Agent.neighbors</code> never populated, <code>callback_url</code> accepted and ignored, <code>gamma_history</code> accumulated and discarded, <code>FlashCrash.duration</code> destructured to <code>—</code>	five files	Individually harmless; collectively they suggest features that were started and abandoned.

	FINDING	WHERE	CONSEQUENCE
14	TypeScript convergence window is 100 samples where Python's is 5	<code>btut-engine.ts:55</code>	The browser engine can rarely evaluate convergence at all.
15	Seventeen root-level markdown files with overlapping and contradictory deployment instructions	<code>repository root</code>	No single source of truth for how to run or deploy the system.

What to do next

An ordered plan. Cheapest and highest-value first.

13.1 First, make the claims true

These four changes are small and they change what the system *is*, not just how it runs.

1. **Fix the update rule.** Replace the symmetric comparison in `_update_strategies` with the logistic form already written in the TypeScript port: `P(switch) = 1/(1 + exp(-γ · wi · Δu))`. This restores per-agent heterogeneity, makes τ meaningful, keeps $O(N)$, keeps the phase transition, and changes no public interface. Apply the identical change in the Rust WASM engine and both domain bridges.
2. **Add one equivalence test.** Run all three engines on the same seed and assert the trajectories agree within tolerance. This is the test whose absence allowed every divergence in Section 12 to happen.
3. **Move the blocking call off the event loop.** `await asyncio.to_thread(simulator.run)`. One line.
4. **Re-baseline the phase transition after the fix.** With heterogeneous agents the transition should become a sigmoid rather than a step, and the calibrator's `transition_sharpness` metric will start reporting something informative instead of always maximal.

13.2 Second, make the market simulator move

The kernel, order book and metrics are all sound and tested. The default configuration is the problem. Set a non-zero `cooperation_bias`, raise `cooperation_noise` so agents start dispersed, and make the shipped default preset one that produces a moving series — the `shock` preset already does. Then raise the kernel decay λ , because at 0.1 the four trader classes adapt at within 10% of each other and the class distinctions the code carefully models never show up in the output.

13.3 Third, connect what already exists

The market simulator is the most sophisticated thing in this repository and nothing can reach it. Two routes on the existing FastAPI service — one to run a market scenario, one to fetch its metrics — would put 5,264 lines of working, tested code in front of users. The `frontend/` application was evidently built for exactly this and is currently serving a static page.

13.4 Fourth, tidy the operational story

1. Resolve the Vercel contradiction: decide whether the Python API deploys there, and make `vercel.json` and `.vercelignore` agree.
2. Pick one of the two Next.js applications and delete or clearly scope the other.
3. Wire the Redis that compose already provisions into the simulation store and rate limiter, or run a single worker and say so.
4. Collapse seventeen root markdown files into one README, one DEPLOYMENT, and an archive directory.

Nothing in Sections 12 and 13 changes the assessment that the underlying idea is sound and the engineering is often good. The mean-field substitution is a legitimate answer to a real complexity problem; the $O(N)$ scaling claim measures out correctly; the order book is properly implemented; the calibrator's bounds check shows real care. The gap is between what the code was designed to do and what one symmetric multiplication currently lets it do — and that gap is a few lines wide.